

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 5

Duration: 1 Hour
Total Marks:50

Annexure A

Scenario Based Assignment

Code of Conduct:

I _____ V. Asha Sarthiwika(192311066)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: V. Asha Sarthiwika

Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

- i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty
- ii. Explain how A* would handle the same situation, including how it balances cost and heuristic
- iii. Analyze the impact of heuristic accuracy on both algorithms
- iv. Discuss how dynamic changes (blocked paths) affect search performance
- v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. Scenario: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

- i. Explain how Hill Climbing would work in this scenario and identify its limitations
- ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation
- iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations
- iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. Scenario: An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

- i. Explain why this problem requires an online search agent instead of offline search

- ii. Analyze the challenges of partial observability and dynamic environments
- iii. Describe how the rover builds and updates its internal model
- iv. Compare online search with uninformed and informed search strategies
- v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

- i. Clearly define variables, domains, and constraints with explanation
- ii. Explain how backtracking search works in this scenario
- iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency
- iv. Analyze real-world challenges and justify the best strategy for scalability

5. **Scenario:** Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must compete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

- i. Explain how the Minimax algorithm models this problem
- ii. Analyze the computational challenges of large game trees
- iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning
- iv. Design an evaluation function and justify the factors considered
- v. Recommend a strategy for balancing optimality and real-time performance

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

1. AI-Powered Drones for Emergency Medicine Delivery

The government uses AI-powered drones to deliver medicines in flood-affected areas. The drone operates in a highly dynamic environment where roads may become blocked unexpectedly, weather changes increase travel cost, and only partial real-time information is available. The drone must reach the destination quickly while minimizing delay and ensuring safety.

i. Greedy Best-First Search (GBFS)

Greedy Best-First Search selects the path that appears closest to the goal using only the heuristic value $h(n)$ (straight-line distance). It ignores the actual travel cost already incurred. Behavior in this scenario

- Chooses the route that appears geographically closest.
- Quickly finds a path if no obstacles exist.
- If a selected route becomes blocked due to flooding, it must search again.
- It may repeatedly choose risky or expensive paths because it ignores terrain and weather costs.

Strengths

- Very fast decision-making.
- Requires less computation.
- Suitable when immediate response is more important than perfect optimization.

Weaknesses

- Does not guarantee the shortest or safest path.
- Can easily get trapped by misleading heuristic estimates.
- Performs poorly when roads frequently become blocked.

ii. A Search Algorithm*

A* evaluates every path using

$$f(n) = g(n) + h(n)$$

where

- $g(n)$ = actual travel cost from the start
- $h(n)$ = estimated distance to the goal

Behavior in this scenario

A* considers both:

- Current travel cost
- Estimated remaining distance

It avoids routes with high travel cost due to flooding or weather, even if they seem geographically shorter.

Advantages

- Produces the optimal path if the heuristic is admissible.
- Considers safety and travel cost.
- Adapts better to varying terrain costs.

Limitations

- Requires more memory.
- Slower than Greedy Search.
- Dynamic obstacles may require replanning.

iii. Impact of Heuristic Accuracy

For Greedy Best-First Search

- Highly dependent on heuristic accuracy.
- Accurate heuristic $\rightarrow$ fast path.

- Poor heuristic → wrong direction and unnecessary exploration.

For A*

- Less sensitive because actual cost is also considered.
- Accurate heuristic speeds up search.
- Poor heuristic increases computation but still generally finds the optimal solution.

Comparison

Accurate Heuristic Greedy	A*
Speed	Very High High
Optimality	Not Guaranteed Guaranteed
Safety	Moderate High

iv. Effect of Dynamic Changes

Floods can suddenly block roads.

Greedy Search

- May continue toward blocked paths.
- Needs frequent restarting.
- Search becomes inefficient.

A*

- Can recalculate the path using updated costs.
- More reliable under changing conditions.
- Better handles partial real-time updates.

v. Recommendation

Recommended Algorithm: A*

Justification

Factor	Greedy	A*
Speed	Excellent	Good
Safety	Medium	Excellent
Optimal Path	No	Yes
Handles Variable Costs	Poorly	Excellent
Real-Time Reliability	Moderate	High

Although Greedy Search is faster, emergency medicine delivery requires safe and reliable routing rather than only speed. A* balances travel cost and heuristic information, making it more suitable for minimizing delay while avoiding dangerous routes.

2. Smart City Traffic Signal Optimization

A smart city controls hundreds of traffic signals to reduce:

- Waiting time
- Fuel consumption
- Traffic congestion

Traffic conditions change continuously, creating an extremely large optimization problem.

Optimization Problem

Objective Function

Minimize

- Total waiting time
- Fuel consumption
- Vehicle congestion

Decision Variables

- Signal timing
- Green light duration
- Signal sequence

Constraints

- Traffic regulations
- Pedestrian crossings
- Emergency vehicle priority
- Maximum and minimum green time

Why Traditional Search is Inefficient

Traditional search explores every possible signal combination.

With hundreds of intersections, the search space becomes enormous.

Time complexity grows exponentially.

Therefore, exhaustive search is impractical.

i. Hill Climbing

Hill Climbing starts with an initial traffic signal configuration.

It repeatedly makes small changes that improve traffic flow.

Example

Increase green time at a congested intersection.

If waiting time decreases, keep the change.

Otherwise discard it.

Advantages

- Simple
- Fast
- Uses little memory

Limitations

- Gets trapped in local optima.
- Cannot explore distant better solutions.

ii. Local Maxima, Plateaus and Ridges

Local Maximum

Traffic improves at one junction but worsens city-wide.

Algorithm incorrectly assumes the solution is optimal.

Plateau

Several timing adjustments produce the same traffic performance.

The algorithm cannot determine which direction is better.

Ridge

The optimal solution requires changing several intersections simultaneously.

Hill Climbing changes only one variable at a time.

It therefore misses better solutions.

iii. Simulated Annealing and Genetic Algorithm

Simulated Annealing

Sometimes accepts worse solutions temporarily.

This helps escape local maxima.

Advantages

- Avoids getting stuck.
- Better exploration.

Genetic Algorithm

Works like biological evolution.

Steps

1. Create population of traffic schedules.
2. Evaluate fitness.
3. Select best schedules.
4. Perform crossover.
5. Apply mutation.
6. Repeat until optimal.

Advantages

- Searches globally.
- Handles huge search spaces.
- Adapts to changing traffic.

iv. Recommendation

Recommended Algorithm: Genetic Algorithm

Justification

Feature	Hill Climbing	Genetic Algorithm
Scalability	Low	High
Dynamic Adaptation	Poor	Excellent
Escapes Local Optima	No	Yes
Multi-objective Optimization	Limited	Excellent

Genetic Algorithms provide better scalability and adaptability, making them ideal for large smart-city traffic optimization.

3. Mars Rover (Online Search)

The rover explores unknown terrain.

It has

- Limited sensors
- No complete map
- Unknown obstacles
- Real-time decision requirements

i. Why Online Search?

Offline search requires a complete map before planning.

Mars terrain is unknown.

The rover discovers new information only while moving.

Therefore, Online Search is essential.

ii. Challenges

Partial Observability

The rover cannot detect distant obstacles.

It only observes nearby terrain.

Dynamic Environment

Dust storms

Moving rocks

Sensor errors

Changing terrain

These require continuous replanning.

iii. Internal Model

The rover maintains an internal map.

Initially

Unknown terrain.

As it moves

- Updates obstacle locations.
- Stores safe paths.
- Marks dangerous regions.
- Improves future navigation.

iv. Online vs Uninformed vs Informed Search

Feature	Uninformed	Informed	Online
Complete Map	Required	Required	Not Required
Uses Heuristic	No	Yes	Optional
Learns While Moving	No	No	Yes
Dynamic Environment	Poor	Moderate	Excellent

v. Recommendation

Best Approach: Online Search Agent

Reasons

- Handles uncertainty.
- Learns continuously.
- Makes safe real-time decisions.
- Suitable for unknown environments.

4. University Exam Timetable (CSP)

Problem Formulation

Variables

Each course exam.

Example

Math

Physics

AI

DBMS

Domains

Possible

- Dates
- Time slots
- Exam halls

Constraints

- No student has overlapping exams.
- Hall capacity must not exceed limit.
- Faculty availability.
- Subject ordering.

- Special accommodation requirements.

i. Variables, Domains and Constraints

Variables represent exam assignments.

Domains represent possible schedules.

Constraints ensure valid timetables.

ii. Backtracking Search

Steps

1. Assign first exam.
2. Check constraints.
3. If valid continue.
4. If violation occurs return (backtrack).
5. Try another assignment.

Eventually a valid timetable is produced.

iii. Constraint Propagation (Forward Checking)

Forward checking removes invalid future choices immediately.

Example

If AI is assigned Monday Morning,

then conflicting courses cannot use Monday Morning.

This reduces unnecessary search.

Advantages

- Faster.
- Fewer backtracks.
- Efficient for large problems.

iv. Challenges and Recommendation

Challenges

- Thousands of students.
- Hall shortages.
- Faculty changes.
- Last-minute updates.

Recommended Strategy

Backtracking + Forward Checking + Constraint Propagation

Reasons

- Faster.
- Highly scalable.
- Produces valid schedules efficiently.

5. Strategic Game AI (Minimax & Alpha-Beta Pruning)

Scenario Understanding

The AI competes against human players.

It must

- Think quickly.
- Evaluate many moves.
- Operate under strict time limits.

i. Minimax Algorithm

Minimax assumes

- AI tries to maximize its score.
- Opponent tries to minimize AI's score.

The algorithm builds a game tree and selects the move with the best worst-case outcome.

ii. Computational Challenges

Branching factor is huge.

Example

35 moves per state.

Depth 8

Nodes $\approx 35^8$

This becomes computationally expensive.

Memory usage also increases.

iii. Alpha-Beta Pruning

Alpha = Best value for Maximizer.

Beta = Best value for Minimizer.

If

$\text{Alpha} \geq \text{Beta}$

then remaining branches cannot improve the result.

They are ignored.

Benefits

- Reduces node evaluation.
- Speeds up decision-making.
- Produces the same optimal move as Minimax.

iv. Evaluation Function

A suitable evaluation function can be:

Score = (Units $\times$ 5) + (Resources $\times$ 3) + (Map Control $\times$ 4) + (Health $\times$ 2) – (Enemy Strength $\times$ 5)

Factors

- Number of surviving units
- Available resources
- Territory controlled
- Unit health
- Enemy strength

These reflect the overall advantage of the AI in the game.

v. Recommendation

Best Strategy

Alpha-Beta Pruning + Evaluation Function + Limited Search Depth (Depth-Limited Minimax)

Justification

Feature	Minimax	Alpha-Beta
Optimal Decision	Yes	Yes
Speed	Slow	Faster
Real-Time Suitable	No	Yes
Scalability	Limited	Better

Alpha-Beta Pruning is the best choice because it preserves the optimal decision quality of Minimax while significantly reducing computation time. Combined with a strong evaluation

function and depth-limited search, it provides a practical balance between optimality, scalability, and real-time performance, making it well suited for modern strategy games.